



Desarrollo Web Profesional

José Miguel Carrera Pacheco



**CARLOS EDUARDO HERNANDEZ
CAMACHO**

8.-A

**Investigación: Arquitectura de
autenticación y control de roles**

08/03/2026

Autenticación vs Autorización

- **Autenticación:** proceso mediante el cual el sistema verifica la identidad del usuario (login).
- **Autorización:** proceso mediante el cual el sistema determina qué acciones puede realizar ese usuario según su rol.

En la arquitectura propuesta:

- Primero se valida identidad.
- Después se validan permisos.

JWT, Sesiones y OAuth

JWT (JSON Web Token)

Token firmado digitalmente que contiene:

- ID de usuario
- Rol
- Fecha de expiración

Es ideal para SPA como React porque:

- Es stateless.
- Funciona bien con APIs REST.
- Escalable.

Sesiones

El servidor guarda estado de sesión.

No es ideal para SPA distribuidas o microservicios.

OAuth

Protocolo para autenticación con terceros como:

- Google
- GitHub

No se implementará en esta fase.

Decisión Técnica

Se eligió:

JWT + RBAC (Role-Based Access Control)

Porque el sistema es:

- SPA con React.
- API desacoplada.
- Posible despliegue en contenedores Docker.

Definición de Usuarios, Roles y Permisos

Tipos de Usuarios

1. ADMIN
2. EMPLEADO
3. CLIENTE

Roles

Rol	Descripción
ADMIN	Control total del sistema
EMPLEADO	Gestión operativa
CLIENTE	Acceso limitado

Permisos

Permiso	ADMIN	EMPLEADO	CLIENTE
Crear registros	SI	SI	NO
Editar registros	SI	SI	NO
Eliminar registros	SI	NO	NO

Permiso	ADMIN	EMPLEADO	CLIENTE
Ver información	SI	SI	SI
Acceso panel admin	SI	NO	NO

Flujo Completo de Autenticación

Login

Usuario → React → API → Validación BD → Generar JWT → React guarda token

Proceso:

- Usuario envía credenciales.
- Backend valida en base de datos.
- Si son correctas:
 - Genera JWT firmado.
 - Incluye rol.
 - Define expiración (ej. 1h).

Acceso a Recursos

Cada petición:

React → Header Authorization: Bearer Token → API → Middleware verifica:

- Firma
- Expiración
- Rol

→ Acceso o rechazo

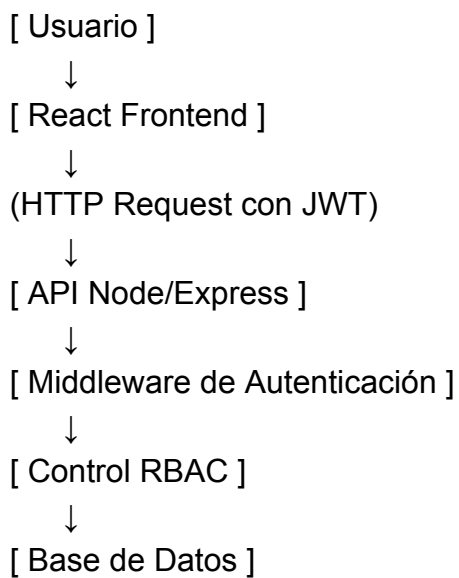
Restricciones

- Sin token → 401 Unauthorized.
- Token inválido → 401.
- Rol sin permiso → 403 Forbidden.

Riesgos de Seguridad Identificados

Riesgo	Impacto	Mitigación
Escalación de privilegios	Acceso indebido	Validación en backend
Token robado	Suplantación	Expiración corta
Manipulación de token	Acceso falso	Firma segura
Validación solo en frontend	Control total del sistema	Backend obligatorio
Fuerza bruta	Robo de cuentas	Rate limiting

Diagrama General de Arquitectura



Separación clara:

- React → solo interfaz.
- Backend → lógica y seguridad.
- BD → almacenamiento.

MI Participación COMO Tester

Garantizar que la arquitectura sea:

- Segura
- Testeable
- Escalable
- Correctamente segmentada por roles

Validaciones realizadas

Como QA se validó que:

- La autenticación esté separada de la autorización.
- El rol esté incluido en el JWT.
- Cada endpoint tenga validación de permisos.
- No exista lógica de autorización solo en frontend.
- Se definan códigos HTTP correctos (401, 403).

Casos de Prueba Diseñados

Autenticación

1. Login correcto → genera token.
2. Login incorrecto → error controlado.
3. Token expirado → acceso denegado.
4. Token modificado manualmente → rechazado.
5. Acceso sin token → 401.

Autorización

6. CLIENTE intenta acceder a ruta ADMIN → 403.
7. EMPLEADO intenta eliminar registro exclusivo de ADMIN → 403.
8. ADMIN accede a todo → permitido.
9. Manipulación de rol desde frontend → sin efecto.
10. Intento de acceso directo vía URL → validado por backend.

Pruebas de Seguridad Planeadas

Se propone uso de herramientas como:

- Postman
- OWASP ZAP
- Burp Suite

Para validar:

- Inyección de token inválido.
- Manipulación de headers.
- Ataques básicos de fuerza bruta.
- Acceso indebido a endpoints.